

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:

Н. Д. Заяц

студент группы ПО-13

Проверил:

А. Д. Кулик

13.02.2026

Цель работы:

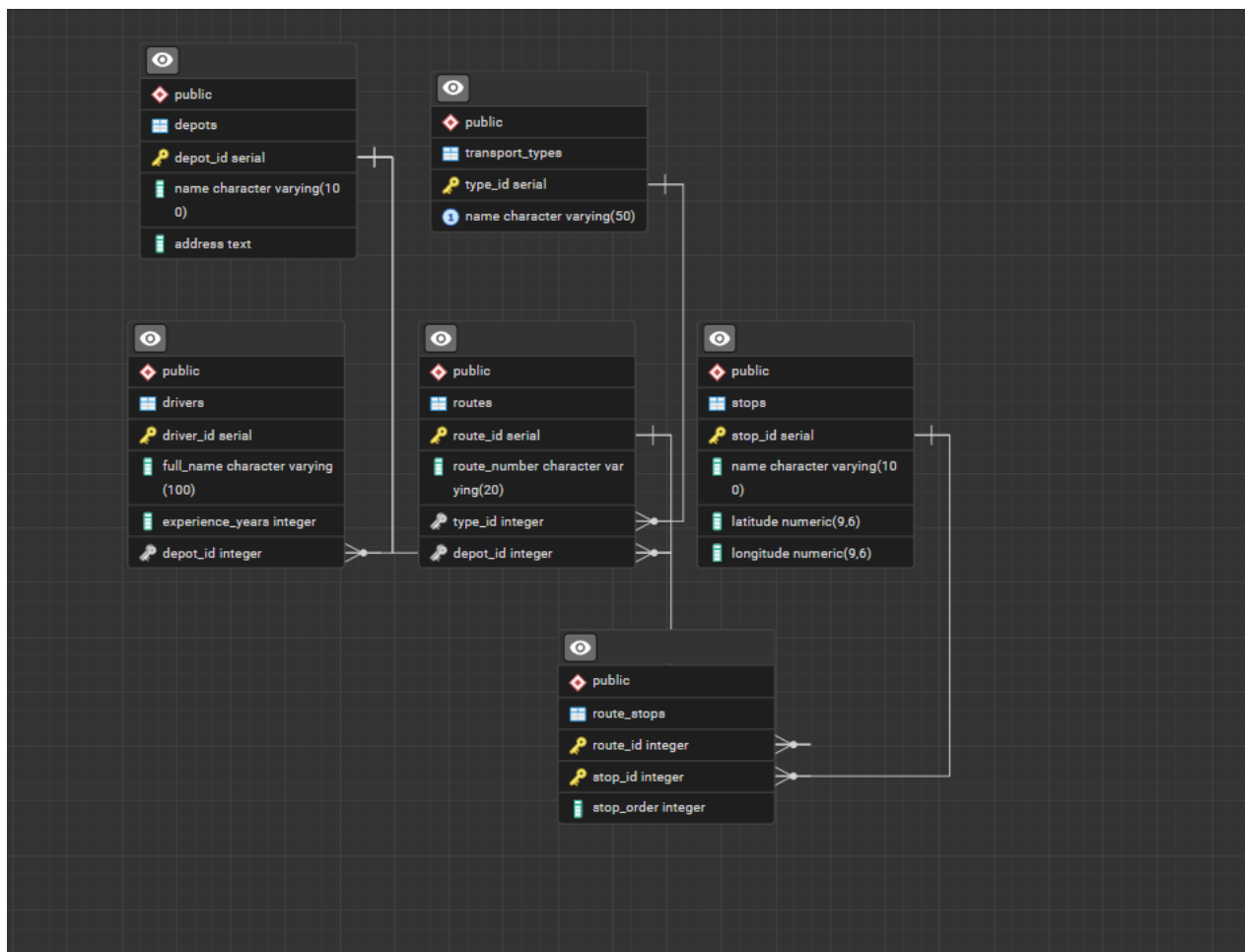
приобрести практические навыки разработки API и баз данных

Общее задание:

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Задание: 4) База данных Городской транспорт

База данных:



Структура проекта:

```
src/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   └── crud.py
└── .env
```

Код программы:

```
"""Основной модуль FastAPI приложения."""

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session

import models
import crud
import schemas

from database import engine, SESSION_LOCAL

# создание таблиц
models.Base.metadata.create_all(bind=engine)

app = FastAPI()

# ----- DB -----
def get_db():
    """Сессия БД."""
    db = SESSION_LOCAL()
    try:
        yield db
    finally:
        db.close()

# Depots

@app.post("/depots/", response_model=schemas.DepotOut)
def create_depot(data: schemas.DepotCreate, db: Session = Depends(get_db)):
    """FastAPI"""
    return crud.create_depot(db, data)

@app.get("/depots/", response_model=list[schemas.DepotOut])
def get_depots(db: Session = Depends(get_db)):
    """FastAPI"""
    return crud.get_depots(db)
```

```

@app.put("/depots/{depot_id}")
def update_depot(depot_id: int, name: str, address: str, db: Session = Depends(get_db)):
    """FastAPI"""
    obj = db.query(models.Depot).filter(models.Depot.depot_id == depot_id).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Not found")

    obj.name = name
    obj.address = address

    db.commit()
    db.refresh(obj)
    return obj

```

```

@app.delete("/depots/{depot_id}")
def delete_depot(depot_id: int, db: Session = Depends(get_db)):
    """FastAPI"""
    obj = db.query(models.Depot).filter(models.Depot.depot_id == depot_id).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Not found")

    db.delete(obj)
    db.commit()
    return {"message": "deleted"}

```

Routes

```

@app.post("/routes/", response_model=schemas.RouteOut)
def create_route(data: schemas.RouteCreate, db: Session = Depends(get_db)):
    """FastAPI"""
    return crud.create_route(db, data)

```

```

@app.get("/routes/", response_model=list[schemas.RouteOut])
def get_routes(db: Session = Depends(get_db)):
    """FastAPI"""
    return crud.get_routes(db)

```

```

@app.put("/routes/{route_id}")
def update_route(route_id: int, route_number: str, type_id: int, depot_id:
    int, db: Session = Depends(get_db)):
    """FastAPI"""
    obj = db.query(models.Route).filter(models.Route.route_id == route_id).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Not found")

    obj.route_number = route_number
    obj.type_id = type_id

```

```
obj.depot_id = depot_id
```

```
db.commit()
```

```
return obj
```

```
@app.delete("/routes/{route_id}")
```

```
def delete_route(route_id: int, db: Session = Depends(get_db)):
```

```
    """FastAPI"""
```

```
    obj = db.query(models.Route).filter(models.Route.route_id == route_id).first()
```

```
    if not obj:
```

```
        raise HTTPException(status_code=404, detail="Not found")
```

```
    db.delete(obj)
```

```
    db.commit()
```

```
    return {"message": "deleted"}
```

```
# Stops
```

```
@app.post("/stops/", response_model=schemas.StopOut)
```

```
def create_stop(data: schemas.StopCreate, db: Session = Depends(get_db)):
```

```
    """FastAPI"""
```

```
    return crud.create_stop(db, data)
```

```
@app.get("/stops/", response_model=list[schemas.StopOut])
```

```
def get_stops(db: Session = Depends(get_db)):
```

```
    """FastAPI"""
```

```
    return crud.get_stops(db)
```

```
@app.put("/stops/{stop_id}")
```

```
def update_stop(stop_id: int, name: str, db: Session = Depends(get_db)):
```

```
    """FastAPI"""
```

```
    obj = db.query(models.Stop).filter(models.Stop.stop_id == stop_id).first()
```

```
    if not obj:
```

```
        raise HTTPException(status_code=404, detail="Not found")
```

```
    obj.name = name
```

```
    db.commit()
```

```
    return obj
```

```
@app.delete("/stops/{stop_id}")
```

```
def delete_stop(stop_id: int, db: Session = Depends(get_db)):
```

```
    """FastAPI"""
```

```
    obj = db.query(models.Stop).filter(models.Stop.stop_id == stop_id).first()
```

```
    if not obj:
```

```
        raise HTTPException(status_code=404, detail="Not found")
```

```
    db.delete(obj)
```

```
    db.commit()
```

```
return {"message": "deleted"}
```

Route Stops

```
@app.post("/route_stops/", response_model=schemas.RouteStopOut)
def create_route_stop(data: schemas.RouteStopCreate, db: Session = Depends(get_db)):
    """FastAPI"""
    return crud.create_route_stop(db, data)

@app.get("/route_stops/", response_model=list[schemas.RouteStopOut])
def get_route_stops(db: Session = Depends(get_db)):
    """FastAPI"""
    return crud.get_route_stops(db)

@app.put("/drivers/{driver_id}")
def update_driver(driver_id: int, full_name: str, depot_id: int, db: Session = Depends(get_db)):
    """FastAPI"""
    obj = db.query(models.Driver).filter(models.Driver.driver_id == driver_id).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Not found")

    obj.full_name = full_name
    obj.depot_id = depot_id

    db.commit()
    return obj

@app.delete("/drivers/{driver_id}")
def delete_driver(driver_id: int, db: Session = Depends(get_db)):
    """FastAPI"""
    obj = db.query(models.Driver).filter(models.Driver.driver_id == driver_id).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Not found")

    db.delete(obj)
    db.commit()
    return {"message": "deleted"}
```

Drivers

```
@app.post("/drivers/", response_model=schemas.DriverOut)
def create_driver(data: schemas.DriverCreate, db: Session = Depends(get_db)):
    """FastAPI"""
    return crud.create_driver(db, data)

@app.get("/drivers/", response_model=list[schemas.DriverOut])
def get_drivers(db: Session = Depends(get_db)):
    """FastAPI"""
```

```
return crud.get_drivers(db)
```

```
@app.delete("/route_stops/")
```

```
def delete_route_stop(route_id: int, stop_id: int, db: Session = Depends(get_db)):
```

```
    """FastAPI"""
```

```
    obj = db.query(models.RouteStop).filter(
        models.RouteStop.route_id == route_id,
        models.RouteStop.stop_id == stop_id
    ).first()
```

```
    if not obj:
```

```
        raise HTTPException(status_code=404, detail="Not found")
```

```
    db.delete(obj)
```

```
    db.commit()
```

```
    return {"message": "deleted"}
```

Спецификация ввода

Введите команду запуска сервера: <строка>

Пример

Введите команду запуска сервера: `uvicorn app.main:app --reload`

Спецификация вывода

Запускаем сервер FastAPI...

Команда: <команда_запуска> Сервер
успешно запущен!

Документация доступна по адресу:

<http://127.0.0.1:8000/docs#/>

Доступные эндпойнты:

- /deports
- /routes
- /stops
- /route Stops
- /drivers

Готово к работе.

Пример

Запускаем сервер FastAPI...

Команда: `uvicorn app.main:app --reload` Сервер
успешно запущен!

Документация доступна по адресу:
<http://127.0.0.1:8000/docs>

Доступные эндпойнты:

- /deports
- /routes
- /stops
- /route Stops
- /drivers

API готово к использованию.

Рисунки с результатами работы программы

```
PS C:\Users\Wwrite\Documents\GitHub\spp_po13\reports\Zayats\5\src> py -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Wwrite\\Documents\\GitHub\\spp_po13\\reports\\Zayats\\5\\src']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [28116] using StatReload
INFO: Started server process [29988]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:54174 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:54174 - "GET /openapi.json HTTP/1.1" 200 OK
```

FastAPI 0.10.0 OAS 3.1
[/openapi.json](#)

default		^
GET	/deports/ Get Depots	▼
POST	/deports/ Create Depot	▼
PUT	/deports/{depot_id} Update Depot	▼
DELETE	/deports/{depot_id} Delete Depot	▼
GET	/routes/ Get Routes	▼
POST	/routes/ Create Route	▼
PUT	/routes/{route_id} Update Route	▼
DELETE	/routes/{route_id} Delete Route	▼
GET	/stops/ Get Stops	▼
POST	/stops/ Create Stop	▼
PUT	/stops/{stop_id} Update Stop	▼
DELETE	/stops/{stop_id} Delete Stop	▼

Вывод: В ходе выполнения работы был разработан и развернут серверный веб-приложение на основе FastAPI, обеспечивающее взаимодействие с реляционной базой данных PostgreSQL через ORM-библиотеку SQLAlchemy.